

One-driver nets

A wire or logic net should have at most one active driver in ordinary RTL

Fight, mux, or tri-state

- Two assigns on the same net both pushing one and zero is the classic fight, net becomes X
- A mux assign `net equals sel question-mark b colon a` is still one driver structurally
- Tri-state uses enables: when off, the driver outputs Z so another may drive
- That is fine on a bus if enables are mutually exclusive
- Two always blocks writing the same reg is the procedural version of the same mistake

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / One-driver / multi-driver

INTERACTIVE TOOL

One-driver / multi-driver

Contested nets resolve to X. RTL wants one driver — mux, enable, or a single procedural block.

Starter example: two assigns drive the same wire — $A=1$ and $B=0 \rightarrow$ resolve **X**. Fix with a mux (one driver) or tri-state with only one enable.

Load starter example

Challenges (1/22)

Quiz: **X**. 0 and 1 both driving a wire resolve to? Answer: X

Your answer

Show hintCheckNextIdle

quiz-xquiz-zquiz-onequiz-muxstarterpreset-fightsafe-zpreset-muxpreset-tri-safe
preset-tri-fightresolveexplainmode-muxmux-sel-bquiz-contendquiz-triactive-1warn-xcode-mux
agree-11floatfull-path

Core ideas

One driver
RTL nets: one assign / one always — or a mux.

Multi-driver
0 vs 1 on a wire $\rightarrow$ X (contention).

Tri-state OK
Multiple assigns allowed if at most one is not Z.

Drivers on the net

Resolution

One-driver lab starter

learn_verilog — one driver

Real Verilog practice

```
wsl - module13-one-driver/examples/navigation

# Track A - real Unix

# real Verilog session (Track A)

$ cat examples/one_driver/one_driver.v
// one_driver.v - one structural driver
module mux_one_driver(
    input sel,
    input a,
    input b,
    output y
);
    assign y = sel ? b : a;
endmodule

// fight_bad - two assigns (commented; sim → X)
// module fight_bad(output wire net);
//     assign net = 1'b1;
//     assign net = 1'b0;
// endmodule

$ iverilog -t null examples/one_driver/one_driver.v
(syntax check passed)
```

Real Verilog - mux as one driver

Pitfalls to watch

- Do not wire two outputs onto the same net without tri-state discipline
- Do not assume synthesis will merge conflicting assigns, it may not match your intent
- Do not use tri-state on internal FPGA fabric when a mux is clearer
- Do not let two clocked always blocks drive the same variable

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser
- On real Verilog, rewrite a contested net using one assign with a mux
- When you are ready